# Lab 5: Frame Detection & Frequency Offset Correction

## Summary

The algorithms for channel estimation and equalization discussed in Lab 4 were formulated under the assumption that the location of the training data was known, which we call the start of the frame. In practice, due to propagation and signal processing delays, the location of the beginning of the frame is unknown. This makes it difficult to apply the channel estimation and equalization algorithms from Lab 4. Up to this point it has also been assumed that the carrier frequency of the transmitter and receiver were locked together. Even small differences, however, lead to a distortion known as carrier frequency offset. In this lab you will implement a particular method for frequency offset correction, called the Moose algorithm, and a method for frame detection based on correlation.

The frequency offset correction and frame detection algorithms you will implement in this lab will be embedded within the synchronization block of the receiver. After building the VIs needed to implement these algorithms, you will verify the functionality and correctness of your code in a simulator. Then, you will implement your code in lab using RF Hardware to see how your code works over a real wireless link.

Note that after this lab you will have considered symbol timing synchronization, frame detection, and channel estimation/equalization all as independent problems. In actuality, any number of these problems could be solved jointly (e.g., jointly solve for the symbol timing offset and channel estimate which minimizes mean-squared error). The task of joint optimization will not be explored in this lab, but you should be aware that the receiver you have been implementing in lab is not necessarily the optimal one. In particular, it is common that frame synchronization is performed jointly with carrier frequency offset estimation.

For the pre-lab submission, you have to turn in the two VIs described in Section 2 (*Moose.vi* and *sliding_correlator.vi*). Further, you have to submit the answers to the questions in Section 2.

## 1 Background

In this section we provide some background on training sequences then review algorithms for frame synchronization and frequency offset synchronization.

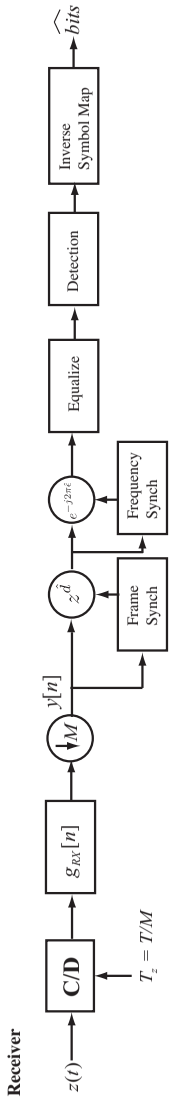



**Figure 1**: The receiver considered in this lab. The new synchronization functionality is introduced after downsampling and before the equalization. The first step is determining the location of the training sequence, known as frame synchronization. The second step is estimation and correction of the carrier frequency offset, known as carrier frequency synchronization. The linear equalizer block includes the channel estimation and equalization functions from Lab 4.

## 1.1 Training Sequences

Up to this point we have assumed that a training sequence has been provided at both the transmitter and receiver. Not all training sequences are created equal. Without a good training sequence, the estimation techniques in the labs would perform poorly. This section discusses some desirable properties of training sequences and introduces one family of sequences known as Barker codes. Many other sequences are used in research literature and in commercial wireless systems as well (e.g., Frank sequences, Zadoff-Chu sequences, or Gold sequences).

Training sequences serve as a known reference signals, which can be used for many purposes at the receiver. In these labs, we use training sequences for two main purposes: (1) synchronization and (2) channel estimation. In general, *good* training sequences for synchronization possess strong autocorrelation properties. In Lab 4, the training sequence needed to be suitably designed in order to provide sufficient rank properties. Training sequences with strong autocorrelation properties can satisfy these rank properties. In this lab, you will use such a training sequence to create a correlation based detector that finds the beginning of a frame.

Barker codes are one set of sequences with good aperiodic autocorrelation and rank properties. A length $N_t$ Barker sequence $\{a_k\}_{k=1}^{N_t}$ is a sequence of values, $\pm 1$, such that

$$\left| \sum_{i=1}^{N_t-k} a_i a_{i+k} \right| \leq 1, \tag{1}$$

for all $1 \leq k \leq N_t$. Because of this strong autocorrelation property, Barker sequences are commonly used in digital communication systems that employ spread spectrum techniques. They have been used for example in IEEE 802.11 and IEEE 802.11b. Table 1 has a list of known Barker codes[1]. The complete set of Barker sequences is the closure of this set of codes under reversal and negation.

The known binary sequences that satisfy Eq. (1) are listed in Table 1. It may happen that longer training sequences are required in a given application. Remember that longer training sequences give better estimation performance. One approach is to relax the condition in Eq. (1) to allow for large cross-correlation values or the binary restriction and thus look for another sequence family. Another approach is to concatenate several Barker sequences together. This will give higher cross-correlation peaks, but provides some other properties that will be exploited for frequency offset estimation.

[1] Shorthand notation is used in this table. "−" and "+" represent −1 and +1 respectively.

| Code Length | Barker Sequence |
|---|---|
| 2 | $[-+, --]$ |
| 3 | $[--+]$ |
| 4 | $[-+--, -+++]$ |
| 5 | $[---+-]$ |
| 7 | $[---++-+]$ |
| 11 | $[---+++-++-+]$ |
| 13 | $[-----++--+-+-]$ |

**Table 1**: Barker Sequences

In the lab we will use a training sequence that consists of four length 11 Barker sequences prepended to the data segment of the packet. This means that you may treat the training sequence as a concatenation of two length 22 training "sub"-sequences (i.e., the complete training sequence is 44 symbols, and the first and second half are the same).

## 1.2 Frame Synchronization

The purpose of frame synchronization is to resolve multiple symbol period delays, determining what we call the beginning of the frame. Essentially the objective is to find a known reference in the transmitted signal so that subsequence receiver operations can proceed like channel estimation and equalization. There is a hugh theory surrounding frame synchronization. We consider one approach based on correlation properties of the training signal. To develop the frame synchronization algorithm we neglect the presence of carrier frequency offset in this section.

Consider a flat-fading channel where symbol synchronization has already been employed to give

$$y[n] = hs[n-d] + v[n]$$

after the matched filter and down-sampling at the receiver where $h$ is an unknown complex channel coefficient and $d$ is an unknown frame offset. Suppose that data is transmitted in frames consisting of a length $N_t$ training signal followed by $P - N_t$ data symbols. Suppose that $\{t[n]\}_{n=0}^{N_t}$ is the training sequence known at the receiver. Since the training sequence has good correlation properties, we correlate the received signal with the training signal to compute

$$R[n] = \left| \sum_{k=0}^{N_t-1} t^*[k]y[n+k] \right|^2 \tag{2}$$

and solve for the frame offset as

$$\hat{d} = \max_n R[n]. \tag{3}$$

The correlation in Eq. (2) results in two terms

$$R[n] = \left| \sum_{k=0}^{N_t-1} t^*[k]hs[n-d] + \sum_{k=0}^{N_t-1} t^*[k]v[n] \right|^2.$$

Because the noise is zero mean, the second term should be approximately zero if the training sequence is large enough. This leads to

$$R[n] \approx \left| \sum_{k=0}^{N_t-1} t^*[k]hs[n+k-d] \right|^2 = |h|^2 \left| \sum_{k=0}^{N_t-1} t^*[k]s[n-d] \right|^2. \tag{4}$$

Assuming that the data is different than the training sequence, the correlation peak should occur at the location of the training data, assuming that the training data is contained in the window of observation samples.

Now suppose that the channel is frequency selective and

$$y[n] = \sum_{\ell=0}^{L} h[\ell]s[n-\ell-d] + v[n] = h[\ell] \star s[n-d] + v[n],$$

where we use $\star$ to denote convolution. In this case, the received correlation $R[n]$ neglecting noise is

$$R[n] \approx \left| h[\ell] \star t^*[-n] \star s[n-d] \right|^2. \tag{5}$$

The effect of the channel is to "smear" the correlation peak. Correlation based frame detection still works, but it becomes less accurate as the peak may shift due to the channel. In practice the peak value of $R[n]$ may be shifted by an offset $\hat{d} = \max_n R[n] - \Delta$ for some $\Delta > 0$ typically $\Delta < L/4$. Equalization takes care of the residual offset.

## 1.3 Frequency Offset Estimation

Passband communication signals are used in wireless communication systems. A passband signal has energy that is nonzero for a frequency band concentrated about some carrier frequency $f_c$. At the transmitter the baseband signal is upconverted to a passband signal while at the receiver a passband signal is downcoverted to a baseband signal. In most systems the carrier frequency used for upconversion at the transmitter does not match the car-

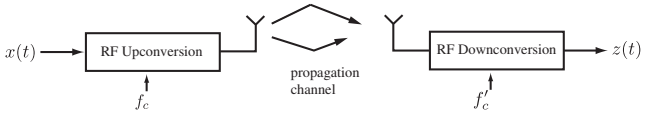



**Figure 2**: Frequency offset between transmitter and receiver.

rier frequency at the receiver, as illustrated in Figure 2. This creates what is known as a carrier frequency offset.

From a baseband signal perspective, the problem of carrier frequency offset can be visualized using the signal model in Figure 3 where the carrier offset is $f_o = f_c - f_c'$. The effect of the offset is to rotate the received signal by $\exp(j2\pi f_o t)$. The larger the offset, the faster the received signal rotates. The carrier offset complicates most of the receiver processing functions including frame synchronization, channel estimation, and equalization.

If the carrier offset is small and sufficient sampling is employed, then the discrete-time signal at the receiver after matched filtering and downsampling can be written as

$$y[n] \quad = \quad e^{j2\pi\epsilon n} \sum_{l=0}^{L} h[l]s[n-l] + v[n],$$

where $\epsilon = f_o T$. The objective of carrier frequency offset synchronization is to estimate the parameter $\epsilon$. Given an estimate $\hat{\epsilon}$, removal of the offset amounts to performing

$$\tilde{y}[n] = e^{-j2\pi\epsilon n} y[n]. \tag{6}$$

To estimate the carrier frequency offset, we use a data aided approach proposed by Moose [7]. This method relies on a periodic training sequence and is applicable to many commercial wireless systems (e.g., IEEE 802.11a/g systems). It a self-referenced synchronization algorithm as it employs the periodic structure in the training but not the correlation properties. Consider

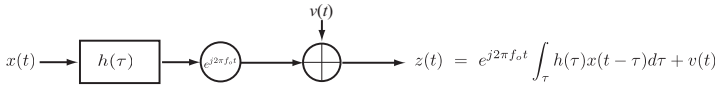



**Figure 3**: Frequency offset between transmitter and receiver.

a complex pulse-amplitude modulation where there is a repetition of two training sequences followed by the data. Let the training sequence start at $n = 0$. Thus

$$s[n] = s[n + N_t] = t[n]$$

for $n = 0, 1, \ldots, N_t - 1$. Note that symbols $s[n]$ for $n < 0$ and $n \geq N_t$ are unknown (they are either zero or correspond to the unknown portions of the data). For $L \leq n \leq N_t - 1$

$$\begin{aligned} y[n] &= e^{j2\pi\epsilon n} \sum_{l=0}^{L} h[l]s[n-l] + v[n] \\ y[n+N_t] &= e^{j2\pi\epsilon(n+N_t)} \sum_{l=0}^{L} h[l]s[n+N_t-l] + v[n+N_t]. \end{aligned}$$

Using the fact that $s[n + N_t] = s[n] = t[n]$ for $n = 0, 1, \ldots, N_t - 1$

$$\begin{aligned} y[n+N_t] &= e^{j2\pi\epsilon N_t} e^{j2\pi\epsilon n} \sum_{l=0}^{L} h[l]t[n-l] + v[n+N_t] \\ &\approx e^{j2\pi\epsilon N_t} y[n]. \end{aligned} \tag{7}$$

To see the significance of this result, remember that the channel coefficients $\{h[l]\}_{l=0}^{L}$ are *unknown*!

One way to solve the frequency offset estimation problem is to formulate and solve a least-squares problem. Because $\epsilon$ appears in the exponent, we solve a modified least squares problem. Consider the squared error

$$J(a) = \sum_{l=L}^{N_t-1} \|y[l+N_t] - ay[l]\|^2.$$

Using the concept of liner least squares from Lab 4, the coefficient $a$ is

$$\hat{a} = \frac{\sum_{l=L}^{N_t-1} y[l+N_t]y^*[l]}{\sum_{l=L}^{N_t-1} |y[l]|^2}.$$

Since only the phase of $\hat{a}$ is of interest, there is no need to compute the denominator. A simple estimate of the frequency offset is then

$$\hat{\epsilon} = \frac{\text{phase} \sum_{l=L}^{N_t-1} y[l+N_t]y^*[l]}{2\pi N_t} \tag{8}$$

or

$$\hat{f}_e = \frac{\text{phase} \sum_{l=L}^{N_t-1} y[l+N_t]y^*[l]}{2\pi T N_t}, \tag{9}$$

where phase denotes the principle phase of the argument.

Thanks to the periodicity of the discrete-time exponential, the estimate of $\epsilon$ will only be accurate for $|\epsilon N_t| \leq \frac{1}{2}$ or equivalently

$$|\epsilon| \leq \frac{1}{2N_t}$$

or

$$|f_e| \leq \frac{1}{2TN_t}.$$

Choosing larger $N_t$ improves the estimate since it results in more noise averaging but reduces the range of offsets that can be corrected. A way of solving this problem is to use multiple repetitions of a short training sequence, but this will not be exploited in this lab. An enhanced carrier frequency offset correction range can be obtained by using properties of the training sequence.

## 1.4 Combining Frame Detection and Frequency Offset Estimation

The frame synchronization algorithm in this lab was developed assuming there was no carrier frequency offset. The presence of frequency offset reduces the accuracy of the correlation based frame detection method. For small offsets this effect can be negligible while for larger offsets, the autocorrelation properties of the training sequence can "fall apart."

The carrier frequency offset estimation algorithm considered in this lab assumed that frame synchronization was already performed. The periodic correlation properties though could be used to perform frame synchronization as well. The observation is that the correlation peak should occur when the pair of training sequences is encountered at the receiver. A robust approach involves evaluating the received correlation in a window and solving

$$\hat{d} = \arg\max \frac{\sum_{n=L}^{N_t-1} y[n+d+N_t]y^*[n+d]}{\sqrt{\sum_{n=L}^{N_t-1} |y[n+d]|^2}\sqrt{\sum_{n=L}^{N_t-1} |y[n+d+N_t]|^2}}. \quad (10)$$

Frame synchronization with the Moose method is robust to intersymbol interference.

In the lab we will employ the training based correlation for frame synchronization followed by the Moose method for carrier frequency offset estimation. It should be clear though that other approaches are possible, including algorithms that jointly perform frame synchronization and carrier frequency offset estimation.

**Table 2**: Description of *Moose.vi*

| *Moose.vi* - implements the Moose algorithm described by Eq. (9) and performs correction per Eq. (6). | | | |
|---|---|---|---|
| **INPUTS** | *input* | 1-D array of CDB (complex doubles) | received sequence after frame detection |
| | *symbol rate* | DBL (double) | symbol rate of the system |
| | *channel estimate length* | I32 (integer) | $L_h + 1$ |
| **OUTPUTS** | *output* | 1-D array of CDB | received sequence after correcting for estimated frequency offset, i.e., $\tilde{y}[n]$ as described by Eq. (6) |
| | *frequency offset* | DBL | frequency offset estimate $\hat{f}_{\text{offset}} = \hat{\epsilon}/T$ |

## 2 Pre-Lab

In this lab you will implement the correlation based frame detector and the Moose algorithm. You will first build *Moose.vi*, that will implement the Moose algorithm. You will then build *sliding_correlator.vi*, which will implement the frame detection algorithm. This VI will also use *Moose.vi* to estimate/correct for the frequency offset. These VIs are described in Tables 2 and 3. After building your VIs, you will insert your code into *frame_detect.vi* in the simulator provided to you and verify your code using the simulator. Please name your VIs in the same manner as your previous submissions (i.e., follow the *yourname_VIname.vi* format).

After building these VIs, insert your code into the simulator provided to you. As shown in Figure 4, you will insert *sliding_correlator.vi* into the appropriate case structure in the block diagram of *frame_detect.vi*.

You can find *frame_detect.vi* in the block diagram of *synchronize.vi*. The hierarchy of these files is shown in Figure 5. After inserting your code appropriately, if you find that the simulator does not compile check to make sure that you have used the same connector topology described by *moose.vi* in the digital communication library.

You have been provided with templates for the VIs you need to create for this lab that already have all the inputs and outputs wired for you. What

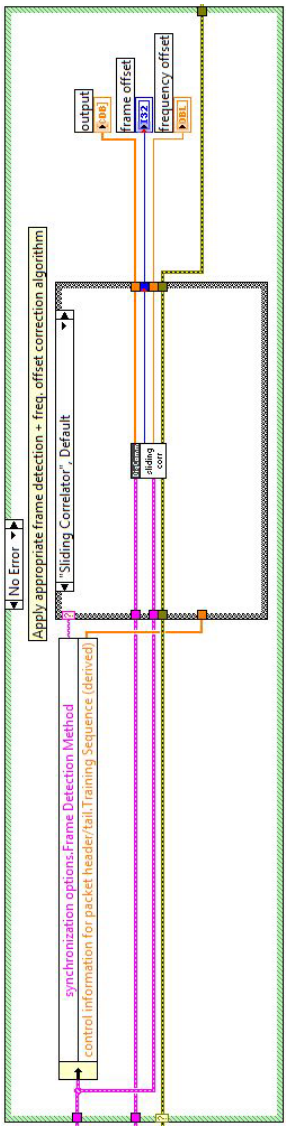



**Figure 4**: Block diagram of *frame_detect.vi*.

**Table 3**: Description of *sliding_correlator.vi*

| *sliding_correlator.vi* - implements the correlation based detector of Eqs. (2)–(3) ; also estimates/corrects for frequency offset using *Moose.vi*. | | | |
|---|---|---|---|
| **INPUTS** | *input complex waveform* | IQ waveform cluster | received sequence after matched filter and symbol timing recovery |
| | *correct frequency offset?* | Boolean | determines whether or not your VI should correct for the frequency offset estimated by *Moose.vi* |
| **OUTPUTS** | *output* | 1-D array of CDB | received sequence after correcting for estimated delay $\hat{d}$ and frequency offset (when appropriate) |
| | *frame offset* | I32 | estimated delay $\hat{d}$ of Eq. (3) |
| | *frequency offset* | DBL | frequency offset estimate computed by *Moose.vi* |

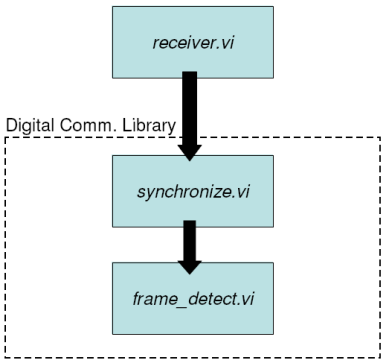



**Figure 5**: Hierarchy of files associated with *frame_detect.vi*.

is required of you is to finish constructing the block diagram to provide the functionality of the VIs.

Throughout the course, several of the VIs you will create will have a *modulation parameters* cluster passed into and out of them. The *modulation*

*parameters in* contains many of the parameters needed by your VIs and will be unbundled from them. This has already been done for you in the template VIs if necessary. Some VIs will also have *modulation parameters out*, so that the cluster can be passed on through to the VIs that follow. Please ensure that these clusters remain wired the way they are in the template VIs, as changing how they are wired will cause VIs further down the line to break.

After building these VIs and inserting them into the simulator as you have done in previous labs, it is important that you verify their functionality. First, verify that your correlation based detector is working by doing the following:

1. Set the *Correct Frequency Offset* control to FALSE on the front panel of the simulator. Setting this constant to FALSE tells *frame_detect.vi* not to correct for the frequency offset estimated by *Moose.vi*.

2. Set the following parameters in your simulator.

   - TX oversample factor = RX oversample factor = 10
   - TX sample rate = RX sample rate = 10 MHz
   - Packet length (bits) = 500
   - Synchronization Method = Timing Estimation
   - ISI Channel = $\{h[0] = 0.35e^{j\pi/4}\}$

   Use the default value for any parameters not specified above.

3. Set the delay of the channel to $10T_s$, where $T_s$ is the symbol period of the system.

After running the top level of your simulator, verify that your receiver has estimated the correct delay in the channel. You can find the measured delay in the *Measured channel impairments* cluster on the front panel of the simulator. Additionally, you should verify that you have no bit errors. After you have verified that your correlation based frame detection algorithm has been implemented properly, you can now check that your implementation of the Moose algorithm is working properly by doing the following:

1. Set the *Correct Frequency Offset* control to TRUE on the front panel of the simulator Setting this constant to TRUE tells *frame_detect.vi* to correct for the frequency offset estimated by *Moose.vi*.

2. Use the same parameters to test your implementation of *Moose.vi*.

3. Set the delay of the channel to $10T_s$, where $T_s$ is the symbol period of the system.

4. Set the frequency offset of the channel to 201 Hz.

Verify that the estimated frequency offset displayed in the *Measured channel impairments* cluster reflects the correct frequency offset. Also, you should have no bit errors. Finally, set the *Correct Frequency Offset* control back to FALSE. Verify that your code estimates the frequency offset, but does not correct for it when this Boolean control is set to FALSE.

In summary:

- After building *Moose.vi* and *sliding_correlator.vi*, insert your code into the simulator (i.e., inside *frame_detect.vi*).

  *Note: You will use the unbundled parameter "Training Sequence" from the modulation parameters in cluster for the training sequence in this lab.*

- Verify that your implementation of the correlation based frame detector and the Moose algorithm are working properly.

## Questions

Answer the following questions, and submit the answers as part of your pre-lab.

1. Set up your simulator according to the steps used for verifying your correlation based frame detector. In particular, make sure that the control *Correct Frequency Offset* is set to FALSE. Set the noise power of your channel to 5 dB. Also, use AWGN channel for this question instead of ISI. Describe what happens to the error rate of your system when the estimate for the delay in the channel is off by more than one symbol time (i.e., when $\hat{d} \neq d$).

   *Note: You may need to modify your VIs to observe this effect, or you may guess the impact from the structure of the frame detection equations.*

2. Set up your simulator according to the steps used for verifying your implementation of the Moose algorithm. Set the *Correct Frequency Offset* control to FALSE. Describe what happens to the received constellation if you do not correct for the 201 Hz frequency offset.

3. Based on the system parameters described in the pre-lab, what are the range of frequency offsets that you can estimate/correct using the Moose algorithm? Since the Moose algorithm cannot estimate/correct for arbitrary frequency offsets it is often considered to perform what is known as fine frequency synchronization.

### Pre-Lab Turn In

1. Submit your implementation of *Moose.vi* and *sliding_correlator.vi*. Remember, you will be penalized if you do not wire the error cluster inputs and outputs in *sliding_correlator.vi* and *Moose.vi*.

   *Note: If you need to generate additional subVIs for this or any future labs, please submit these subVIs along with your other pre-lab VIs.*

2. Submit your answers to all of the questions in the pre-lab section.

## 3 Lab Experiment

In this lab you will run your implementation of *sliding_correlator.vi* over a real wireless link. Using your code from the prior labs in conjunction with the new blocks you have created, you will complete the framework for the transmitter and receiver blocks in lab as in previous labs. Insert your code into this framework as you did in the pre-lab. Next, set up the following parameters in your system.

- TX sample rate = 4 MSamp/sec
- TX oversample factor = 20
- RX sample rate = 4 MSamp/sec
- RX oversample factor = 20
- Channel estimate length = 2
- Equalizer length = 4
- Capture time = 2 msec

Leave any unspecified parameters set to their default values. Also make sure that the *Correct Frequency Offset* control on *top_rx* is set to TRUE. You will now perform two experiments to explore frequency offsets and frame detection over a real wireless link. Before you begin, estimate the order of magnitude of the frequency offset between the transmitter and receiver of your system. Take note of this value as you may be asked about it later.

### 3.1 Correlation Based Frame Detection Under Large Frequency Offsets

The correlation based detector discussed in this lab can be impacted by a number of impairments, including frequency offset, as discussed previously.

Answer the following questions to explore the relationship between frequency offset and your correlation based detector.

1. Before changing any of the settings, calculate an average value (using five runs or so) of the inherent offset between the transmitter and receiver.

2. Based on the system parameters, what is the range of frequency offsets that can be estimated by your frequency offset estimation algorithm?

3. Let $f_M$ be the maximum correctable frequency offset of your system (i.e., as calculated in the previous question). Modify the carrier frequency of your transmitter so that you will cause an effective offset of $0.80\,f_M$ at the receiver. What is the new value of the carrier frequency at your transmitter?

4. How does this frequency offset impact the performance of your correlation based frame detector (i.e., does your correlation based detector still find the start of the frame)? Describe what happens to the error rate and constellation of your system.

You will include your answers to these questions in your lab report. *Note that the frequency offset $0.80\,f_M$ is two orders of magnitude greater than the inherent offset between the transmitter and receiver*.

The correlation based detector discussed in this lab can be impacted by a number of impairments, including frequency offset, as discussed previously. Answer the following questions to explore the relationship between frequency offset and your correlation based detector.

1. Based on the system parameters, what is the range of frequency offsets that can be estimated by your frequency offset estimation algorithm?

2. Let $f_M$ be the maximum correctable frequency offset of your system (i.e., as calculated in the previous question). Modify the carrier frequency of your transmitter so that you will cause an effective offset of $0.80\,f_M$ at the receiver. What is the new value of the carrier frequency at your transmitter?

3. How does this frequency offset impact the performance of your correlation based frame detector (i.e., does your correlation based detector still find the start of the frame)? Describe what happens to the error rate and constellation of your system.

You will include your answers to these questions in your lab report.

## 3.2 Self referenced frame synchronization

In Section 3.1, you saw how the correlation based detector can "break" under large frequency offsets. You will now explore another frame detection algorithm which is more robust to frequency offsets. As discussed in class, a self referenced frame synchronization method can be derived from the least-squares expression of the Moose algorithm. This frame detection performs the calculation in Eq. (10). This frame detection method has already been implemented in *digital_comm3.2b.llb*. Change the frame detection method of your system by modifying the appropriate parameter on the front panel of *top_rx.vi*. Observe how the *Self Reference Frame Synchronization* method performs under the large frequency offset computed in the last section.

Consider a simple flat fading channel model with frequency offset $f_{\text{offset}} = \epsilon/T$ ($T$ is the symbol period of the system) such that the received sequence is given by

$$y[n] = \alpha\, e^{j2\pi\epsilon n}\, t[n - D], \tag{11}$$

where $t[n]$ is the transmitted training sequence, $\alpha$ is a complex scalar, and $D$ is the delay introduced by the channel. Include the answer to the following question in your lab report:

**Question:** Using the simple channel model of Eq. (11), explain why the self referenced frame detector of Eq. (10) works better than the correlation based detector under large frequency offsets. You may also need to consider the mathematical description of the correlation based detector given in Eq. (2). *Hint: Notice that the training sequence is periodic (i.e., $t[n + W] = t[n]$ for all $n \in \{0, 1, \ldots, W - 1\}$). Also you should notice that the output of the frame detection algorithm should be D for this particular channel model.*

## 3.3 Lab Turn In

Demonstrate to the instructor that your code is working properly. Show the instructor that your correlation based detector "breaks" under a large frequency offset (as discussed in Section 3.1). Then show the instructor that under this same frequency offset, the self referenced frame synchronizer works. Also answer all of the questions above. You will be required to submit these answers in a lab report.

## DIRECTIONS FOR LAB SUBMISSION

Your lab report should include the following.

1. Answer all questions from the lab experiment (i.e., do not reanswer pre-lab questions).

2. Discuss any problems you might have encountered and how you overcame these obstacles.

3. In this lab you observed how your code operates over a real wireless link. Answer the following questions about your experience in lab.

   (a) Name three impairments in a real wireless system which impact the accuracy of the correlation based detection method.

   (b) Under large frequency offsets, the correlation based detector "breaks". What property of the training sequence breaks down under large frequency offsets? *Hint: You may need to consider your answer to the question in Section 3.2*.

   (c) In the self referenced frame synchronization method, is it still critical that the periodic training sequence have good autocorrelation properties? If so, why? Use the simple channel model of Eq. (11) to help answer this question, if necessary.

# Lab 6: OFDM Modulation & Frequency Domain Equalization

## Summary

In this lab you will implement the key features of the orthogonal frequency division multiplexing (OFDM) multicarrier modulation technique. OFDM is a transmission technique with a special structure that permits low complexity linear equalization at the receiver. Several commercial wireless systems have adopted OFDM modulation including wireless LAN standards like IEEE 802.11g, IEEE 802.11a, and IEEE 802.11n; broadband wireless access including IEEE 802.16 (WiFi); mobile broadband wireless known as IEEE 802.20; digital video broadcasting DVB (used in Europe); as well as several releases of the 3GPP cellular standards. The OFDM system considered in this lab is illustrated in Figure 1.

Although the design of an OFDM system has several important differences from the single carrier system considered in previous labs, it must still perform many of the same functions (e.g., channel estimation, equalization, and frequency offset estimation). In this lab you will still perform all functions related to synchronization and channel estimation in the time domain using the same training sequence discussed in Lab 5. The framing structure considered in this lab is illustrated in Figure 2.

For the pre-lab submission, you have to turn in the three VIs described in Section 2 (*OFDM_modulate.vi*, *OFDM_demodulate.vi*, and *FEQ.vi*). Further, you have to submit the answers to questions in Section 2.

## 1 Background

### 1.1 OFDM Modulation

Figure 1 shows a block diagram of the basic OFDM modulator you will be implementing in this lab. OFDM is a type of digital modulation where information is modulated into discrete-time sinusoids. With OFDM, the symbols after the constellation mapping are considered to start in the frequency domain. OFDM operates on groups of symbols, the resulting group of symbols being called an OFDM symbol. To begin the explanation, we assume $K = 0$ and that there are $N$ frequency-domain symbols in one OFDM symbol. The number of subcarriers is given by $N$, typically a power of 2. The transmitter operates as follows. Given $\{s[n]\}_{n=0}^{N-1}$ and cyclic prefix of length $L_c$, the